

**Правительство Российской Федерации
Федеральное государственное автономное
образовательное учреждение высшего образования
«Национальный исследовательский университет «Высшая школа
экономики»**

Факультет компьютерных наук
Образовательная программа бакалавриата 09.03.04 «Программная инженерия»

ОТЧЕТ

по производственной практике

Факультет компьютерных наук, образовательная программа «Программная
инженерия»

в (на) Национальный исследовательский университет «Высшая школа
экономики»

Руководитель практики от профильной организации

Профессор, доктор физико-

математических наук

(должность)

Дата:

(подпись)

Попов В. Ю.

(фамилия,
инициалы)

Задание принято к исполнению

Студент

группы БПИ 235

15.05.2026 г.

(подпись)

Лившиц Л. И.

(фамилия,
инициалы)

СОГЛАСОВАНО

Руководитель практики от НИУ ВШЭ:

Руководитель департамента ПИ ФКН,

к.э.н., доцент

Должность, место работы

Дата:

(подпись)

Лебедев С. А.

И.О.Фамилия

Москва - 2026

АННОТАЦИЯ

Производственная практика проходила в период с 1 июля 2026 года по 14 июля 2026 года в рамках проекта по развитию серверной части веб-приложения для обучения языку программирования C# с элементами геймификации. Работа была направлена на повышение устойчивости и наблюдаемости системы: запуск сервисов в Kubernetes, внедрение Celery для управляемой фоновой обработки задач, настройку Prometheus-метрик и проведение нагрузочных испытаний. В результате были подготовлены Kubernetes-манифесты, реализован слой фоновых задач, добавлены метрики API и рабочих процессов, выполнены проверки автомасштабирования и отказоустойчивости.

СОДЕРЖАНИЕ

1. ЦЕЛЬ И ЗАДАЧИ ПРАКТИКИ	4
1.1. Цель	4
1.2. Задачи	4
2. ОПИСАНИЕ МЕСТА ПРОХОЖДЕНИЯ ПРАКТИКИ	5
3. ОБЗОР ИЗУЧЕННЫХ МАТЕРИАЛОВ, ТЕХНОЛОГИЙ И МЕТОДОВ	6
3.1. Серверная часть и API	6
3.2. Фоновые задачи	6
3.3. Контейнеризация и Kubernetes	7
3.4. Наблюдаемость и нагрузочное тестирование	7
4. ОПИСАНИЕ МЕТОДОВ, ТЕХНОЛОГИЙ И СРЕДСТВ РАЗРАБОТКИ, ИСПОЛЬЗОВАННЫХ ДЛЯ РЕШЕНИЯ ПОСТАВЛЕННЫХ ЗАДАЧ	8
4.1. Фиксация исходных замеров перед изменениями	8
4.2. Оптимизация отправки теста и вынос фоновых операций	8
4.3. Внедрение Celery	9
4.4. Добавление Prometheus-метрик	11
4.5. Подготовка Kubernetes-манифестов	12
4.6. Настройка автомасштабирования и отказоустойчивости	12
5. ОПИСАНИЕ ПОЛУЧЕННЫХ РЕЗУЛЬТАТОВ	14
5.1. Результаты исходных замеров	14
5.2. Результаты оптимизации отправки теста	14
5.3. Результаты внедрения Celery	15
5.4. Результаты Kubernetes-развертывания	15
5.5. Результаты нагрузочного тестирования Kubernetes	16
5.6. Сравнение с исходным состоянием	17
6. ЗАКЛЮЧЕНИЕ	18

ТЕРМИНОЛОГИЯ	19
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	20
ПРИЛОЖЕНИЕ	21
Рабочий план-график прохождения практики	21

1. ЦЕЛЬ И ЗАДАЧИ ПРАКТИКИ

1.1. Цель

Целью производственной практики являлось исследование и внедрение инфраструктурных решений для повышения надежности, масштабируемости и наблюдаемости серверной части веб-приложения для обучения языку программирования C# с элементами геймификации.

Практическая цель работы состояла в том, чтобы перевести существующую серверную систему от локального стенда Docker Compose к подходу, близкому к промышленной эксплуатации: выделить фоновые задачи в управляемый слой Celery, подготовить Kubernetes-манифесты для API-сервиса, рабочего процесса, клиентской части, nginx, PostgreSQL и Redis, добавить метрики Prometheus и подтвердить результат нагрузочными проверками.

1.2. Задачи

1. Проанализировать текущую архитектуру проекта и выделить узкие места по производительности и масштабированию.
2. Подготовить контейнеризацию серверных сервисов и описания развертывания в Kubernetes.
3. Внедрить Celery для выноса тяжелых операций в фоновые очереди.
4. Настроить мониторинг приложения и инфраструктуры с использованием Prometheus.

2. ОПИСАНИЕ МЕСТА ПРОХОЖДЕНИЯ ПРАКТИКИ

Производственная практика проходила в рамках образовательной программы бакалавриата 09.03.04 «Программная инженерия» Факультета компьютерных наук НИУ ВШЭ. Работа выполнялась на базе учебного проекта «Веб-приложение для обучения языку программирования C# с элементами геймификации».

В рамках практики объектом работы выступала серверная часть веб-приложения. Система предназначена для организации учебного процесса по языку C#: пользователи изучают материалы, проходят тесты, получают баллы, достижения и элементы геймификации. Преподаватели и администраторы управляют учебным контентом, тестами, группами, аналитикой и AI-геймификацией заданий через административную панель.

Серверная часть проекта реализована как модульное FastAPI-приложение. В системе используются PostgreSQL для хранения данных, Redis для кеширования и очередей, Celery для фоновых задач, Prometheus для метрик, Docker Compose для локального запуска и Kubernetes для инфраструктурного развертывания.

3. ОБЗОР ИЗУЧЕННЫХ МАТЕРИАЛОВ, ТЕХНОЛОГИЙ И МЕТОДОВ

Перед выполнением практической части был проведен обзор технологий, необходимых для контейнеризации, фоновой обработки задач, мониторинга и нагрузочного тестирования серверного приложения.

3.1. Серверная часть и API

- FastAPI использовался как основной веб-фреймворк для реализации REST API. Он поддерживает асинхронную обработку запросов, интеграцию с Pydantic-схемами и автоматическую генерацию OpenAPI-документации.
- SQLAlchemy применялся как ORM-слой для работы с PostgreSQL. Он позволил описывать модели предметной области, управлять связями между сущностями и выполнять запросы в рамках транзакций.
- PostgreSQL использовался как основное реляционное хранилище. В системе хранятся пользователи, материалы, тесты, вопросы, попытки, ответы, баллы, награды, достижения, группы, задания AI-геймификации и служебные сущности.
- Redis выполнял несколько ролей: кеш часто запрашиваемых данных, посредник сообщений для фоновых задач, хранилище метрик очередей и промежуточный слой для снижения нагрузки на PostgreSQL.

3.2. Фоновые задачи

- Celery был изучен и внедрен как механизм управляемой фоновой обработки задач. В отличие от ручных очередей на списках Redis, Celery предоставляет маршрутизацию задач по очередям, повторный запуск при ошибке, задержку между повторными попытками, ограничения времени выполнения, статусы выполнения и удобную интеграцию с рабочими процессами.
- Для проекта были выделены очереди answers, grading, ai и analytics, что позволило разделить разные типы нагрузки: обработку завершенных попыток, проверку открытых ответов, AI-геймификацию и обновление аналитики.

3.3. Контейнеризация и Kubernetes

- Docker использовался для упаковки серверной части, клиентской части и рабочего процесса в воспроизводимые контейнеры. Docker Compose применялся для локального запуска связки серверного приложения, клиентской части, PostgreSQL, Redis, рабочего процесса, nginx и Prometheus.
- Kubernetes был выбран как средство развертывания и масштабирования системы. Были изучены сущности Deployment, Service, ConfigMap, Secret, Job, PersistentVolumeClaim, HorizontalPodAutoscaler и PodDisruptionBudget.
- Проверки готовности и работоспособности использовались для контроля состояния сервисов. Проверка готовности защищает систему от отправки запросов на еще неподнятый pod, а проверка работоспособности позволяет Kubernetes перезапускать зависшие или неисправные контейнеры.

3.4. Наблюдаемость и нагрузочное тестирование

- Prometheus использовался как формат и подход для сбора метрик. В проекте были добавлены метрики API, очередей и Celery-задач.
- Нагрузочное тестирование проводилось для проверки операций чтения, отправки теста и поведения Kubernetes под нагрузкой. Основными метриками стали запросы в секунду, p50, p95, p99, количество ошибок, нагрузка на процессор и изменение числа экземпляров приложения.
- Для проверки автомасштабирования применялся metrics-server в Kubernetes и HorizontalPodAutoscaler, реагирующий на нагрузку процессора у серверного приложения и рабочего процесса.

4. ОПИСАНИЕ МЕТОДОВ, ТЕХНОЛОГИЙ И СРЕДСТВ РАЗРАБОТКИ, ИСПОЛЬЗОВАННЫХ ДЛЯ РЕШЕНИЯ ПОСТАВЛЕННЫХ ЗАДАЧ

4.1. Фиксация исходных замеров перед изменениями

Первым этапом была зафиксирована исходная точка состояния системы. Это было необходимо, чтобы дальнейшие изменения можно было оценивать по измеримым результатам.

В Docker Compose были подняты основные сервисы: серверное приложение, клиентская часть, nginx, PostgreSQL, Redis и рабочий процесс.

Были проведены следующие проверки:

- точки проверки состояния /health/live и /health/ready;
- полный прогон тестов серверной части;
- проверка покрытия кода тестами;
- нагрузочная проверка операций чтения для основных пользовательских страниц;
- отправка теста на 10 закрытых вопросов;
- сравнение текущей отправки одним запросом с более старым подходом «N x POST /answers + POST /complete», который применялся на раннем этапе разработки веб приложения.

На этапе исходных замеров было зафиксировано, что тесты серверной части проходят успешно: 236 passed, покрытие кода тестами составляет 70%. Операции чтения после прогрева кеша стабильно выдерживали 8 и 20 запросов в секунду без ошибок. Это позволило сфокусировать дальнейшую работу не на базовой корректности API, а на управляемости фоновых задач и инфраструктурной устойчивости.

4.2. Оптимизация отправки теста и вынос фоновых операций

Отправка теста на проверку является одним из наиболее важных сценариев записи данных. Ранее проблемным местом был подход, при котором

клиентская часть отправляла ответы по одному запросу, а затем отдельно завершала попытку:

Старый сценарий: «N x POST /answers + POST /complete».

Такой процесс приводил к большому количеству сетевых запросов, повторной валидации и большому числу SQL-операций. По предыдущему аудиту старый сценарий занимал 46.48s и выполнял 12579 SQL-запросов.

В проекте использовалась единая точка API для отправки всех ответов сразу:

Новая точка API: «POST /api/v1/tests/attempts/{attempt_id}/submit».

В синхронной части отправки теста были оставлены только критичные действия:

- проверка доступа пользователя к попытке;
- проверка дедлайна и ограничения времени;
- загрузка вопросов, вариантов и существующих ответов одним проходом;
- сохранение ответов;
- расчет базового результата;
- закрытие попытки;
- постановка фоновых задач после сохранения транзакции.

Вторичные операции были вынесены в фон: начисление наград, события испытаний, обработка открытых ответов, обновление аналитики и дополнительная инвалидация кеша. Это позволило не задерживать ответ пользователю операциями, которые не являются необходимыми для немедленного отображения результата попытки.

4.3. Внедрение Celery

До внедрения Celery фоновые операции частично выполнялись через ручные очереди на списках Redis. Такой подход уже позволял не держать часть работы в HTTP-запросе, но был менее удобен с точки зрения эксплуатации:

повторный запуск при ошибке, маршрутизация, метрики и лимиты выполнения задач требовали ручной реализации.

Для решения этой проблемы был добавлен слой Celery. Redis использовался как посредник сообщений и хранилище результатов. В конфигурации Celery были настроены:

- JSON-сериализация задач;
- подтверждение задачи после выполнения;
- маршрутизация задач по очередям;
- мягкое и жесткое ограничение времени выполнения;
- отслеживание начала выполнения задачи;
- число параллельно выполняемых задач рабочего процесса;
- получение рабочим процессом только одной новой задачи заранее;
- автоматический повторный запуск с увеличением задержки и случайным сбросом времени.

Были выделены четыре основные Celery-задачи:

Задача	Назначение	Очередь
answers_postprocess	обновление наград, испытаний и вторичной аналитики после завершения попытки	answers
open_answer_grading	фоновая обработка открытых ответов и пересчет результата	grading
ai_gamification	генерация AI-драфтов для геймификации материалов и вопросов	ai
analytics_refresh	обновление и инвалидация аналитических данных	analytics

Для сохранения совместимости был оставлен запасной путь на старые очереди Redis. Если в настройках указан механизм фоновых задач redis, система использует прежний способ. Если указан celery, задачи отправляются через Celery. Такой подход снизил риск миграции и позволил внедрять новый механизм постепенно.

4.4. Добавление Prometheus-метрик

Для наблюдаемости системы были добавлены Prometheus-совместимые метрики. Они позволяют анализировать состояние приложения не по логам вручную, а через агрегированные показатели.

В проекте были выделены следующие группы метрик:

- количество HTTP-запросов по маршруту, методу и статусу;
- время ответа API;
- глубина Redis/Celery очередей;

- количество Celery-задач по статусам started, succeeded, failed;
- суммарное, среднее и максимальное время выполнения Celery-задач.

4.5. Подготовка Kubernetes-манифестов

Для запуска системы в Kubernetes были подготовлены манифесты в директории infra/k8s/base. В состав вошли:

- Namespace;
- ConfigMap для конфигурации приложения;
- Secret для чувствительных параметров;
- Deployment и Service для серверного приложения;
- Deployment и Service для рабочего процесса;
- Deployment и Service для клиентской части;
- Deployment и Service для nginx;
- Deployment и Service для Redis;
- Deployment, Service и PersistentVolumeClaim для PostgreSQL;
- Job для выполнения миграций базы данных;
- HorizontalPodAutoscaler для серверного приложения и рабочего процесса;
- PodDisruptionBudget для серверного приложения, рабочего процесса и nginx.

Серверное приложение и рабочий процесс были разделены как разные Deployment, поскольку API и фоновые задачи имеют разные профили нагрузки. API должен быстро отвечать пользователю, а рабочий процесс может масштабироваться отдельно в зависимости от очередей или нагрузки процессора.

4.6. Настройка автомасштабирования и отказоустойчивости

Для серверного приложения был настроен HorizontalPodAutoscaler с минимальным числом экземпляров 2, максимальным числом экземпляров 6 и целевой нагрузкой процессора 65%. Для рабочего процесса был настроен HPA

с минимальным числом экземпляров 1, максимальным 4 и целевой нагрузкой процессора 70%.

Также были добавлены `PodDisruptionBudget`. Они нужны для того, чтобы при добровольных пересозданиях pod или обслуживании кластера Kubernetes не удалил все экземпляры критичного сервиса одновременно.

Для проверки отказоустойчивости выполнялось ручное удаление pod серверного приложения. Deployment автоматически создавал новый pod, проверка готовности защищала от отправки запросов на неподнятый экземпляр, а nginx продолжал направлять запросы на доступные экземпляры.

5. ОПИСАНИЕ ПОЛУЧЕННЫХ РЕЗУЛЬТАТОВ

5.1. Результаты исходных замеров

На этапе исходных замеров были получены следующие результаты:

Проверка	Результат
Полный прогон тестов серверной части	236 passed, 0 failed
Покрытие серверного кода тестами	70%
Операции чтения, 8 запросов/с	0 ошибок, p95 18.24 ms
Операции чтения, 20 запросов/с	0 ошибок, p95 23.13 ms
Отправка теста на 10 закрытых вопросов	0.98s
Health после нагрузки	200 OK

Эти результаты показали, что локальный стенд Docker Compose стабильно работает после прогрева кеша. Основной риск находился не в операциях чтения, а в тяжелых сценариях записи данных и инфраструктурной устойчивости.

5.2. Результаты оптимизации отправки теста

Сравнение старого и нового подхода к отправке теста показало существенное улучшение:

Сценарий	Время	SQL
Старый сценарий: «N x POST / answers + POST /complete»	46.48s	12579
Первый аудит отправки одним запросом: «POST /attempts/{id}/submit»	7.56s	1843
Текущий замер: 10 ответов одним запросом	0.98s	не снимался

Переход к отправке одним запросом уменьшил время первого сравнимого сценария примерно на 83.7%, а количество SQL-запросов примерно на 85.3%.

Celery не заменил эту точку API, а усилил архитектуру вокруг нее: вторичные операции стали выполняться в управляемом фоне.

5.3. Результаты внедрения Celery

После внедрения Celery фоновые задачи получили отдельные очереди, политику повторного запуска, ограничения времени выполнения и метрики. Это повысило управляемость системы и снизило риск того, что тяжелая вторичная операция будет блокировать пользовательский запрос.

Практический результат внедрения Celery состоит в следующем:

- основной HTTP-запрос отвечает пользователю после выполнения критичной бизнес-логики;
- тяжелые операции выполняются рабочим процессом;
- задачи разделены по типам нагрузки;
- рабочий процесс можно масштабировать независимо от серверного приложения;
- ошибки фоновых операций не приводят к падению пользовательского запроса;
- появилась возможность наблюдать количество и длительность задач через метрики.

5.4. Результаты Kubernetes-развертывания

Kubernetes-манифесты были применены на локальном Docker Desktop Kubernetes. В результате были успешно подняты следующие компоненты:

- серверное приложение;
- клиентская часть;
- nginx;
- PostgreSQL;
- Redis;
- рабочий процесс;
- задача миграции базы данных.

Проверка через nginx показала, что приложение доступно снаружи, точка проверки состояния возвращает 200 OK, а API и документация Swagger открываются через единую точку входа.

5.5. Результаты нагрузочного тестирования Kubernetes

Для проверки Kubernetes была добавлена отдельная нагрузочная задача, которая выполнялась внутри кластера по пути:

Путь проверки: задача нагрузки -> служба nginx -> служба серверного приложения.

Результат прогона:

Метрика	Значение
Длительность	60s
Параллельные запросы	25
Всего запросов	21828
Ошибки	0
Запросов в секунду	363.80
Среднее время ответа	71.86 ms
p95	126.62 ms
p99	198.65 ms

После нагрузки НРА обнаружил превышение целевой нагрузки процессора у серверного приложения и увеличил количество pod с 2 до 4. Это подтвердило, что автомасштабирование было не только описано в манифестах, но и реально сработало под нагрузкой.

5.6. Сравнение с исходным состоянием

Исходные отчеты и данные испытаний показывали, что серверная часть работает стабильно в Docker Compose и выдерживает нагрузку операций чтения на 8 и 20 запросов в секунду после прогрева кеша. Однако на этом этапе еще не проверялись Kubernetes, НРА и отказоустойчивость pod.

Итоговые испытания отличаются следующим:

Область	Исходные отчеты и логи	Итоговые испытания
Среда	Docker Compose	Kubernetes Docker Desktop
Фоновые задачи	очереди Redis / рабочий процесс	Celery + Redis + рабочий процесс
Метрики	базовые логи и метрики запросов	точка /metrics + метрики Celery
Автомасштабирование	не проверялось	серверное приложение 2 -> 4 pod
Отказоустойчивость	не проверялась	перезапуск pod и состояние 200 OK
Нагрузочная проверка Kubernetes	не было	363.80 запросов/с, 0 ошибок

Таким образом, исходные отчеты подтвердили скорость серверной части на локальном стенде, а итоговые испытания подтвердили инфраструктурную устойчивость серверной части в Kubernetes.

6. ЗАКЛЮЧЕНИЕ

В ходе производственной практики были выполнены поставленные задачи по исследованию и внедрению инфраструктурных решений для серверной части веб-приложения обучения C# с элементами геймификации.

Были изучены и применены технологии Kubernetes, Celery, Redis, Prometheus, Docker Compose, FastAPI, PostgreSQL и nginx. Работа включала не только написание конфигураций, но и проверку их практической работоспособности через тесты, метрики, нагрузочные испытания и ручные сценарии отказа.

Основной инженерный результат заключается в том, что серверная часть была подготовлена к более устойчивому способу эксплуатации. API и рабочий процесс были разделены, тяжелые вторичные операции вынесены в управляемые задачи Celery, добавлены Prometheus-метрики, описаны Kubernetes-манифесты, настроены HPA и PDB.

Практические измерения подтвердили результат: операции чтения на 20 запросах в секунду проходили без ошибок с p95 23.13 ms, текущая отправка 10 закрытых вопросов одним запросом выполнялась примерно за 0.98s, а нагрузочное испытание Kubernetes показало 363.80 запросов в секунду без ошибок и вызвало автоматическое масштабирование серверного приложения с 2 до 4 pod.

Направления дальнейшей работы включают подключение полноценной панели Grafana, нагрузочное тестирование полного бизнес-сценария отправки теста в Kubernetes, масштабирование рабочего процесса по глубине очередей через KEDA, проверку поведения системы при отказе Redis или PostgreSQL, а также подготовку профиля разворачивания для промышленной среды.

ТЕРМИНОЛОГИЯ

Термин	Используемое значение
Серверная часть	Компонент приложения, который обрабатывает HTTP-запросы, выполняет бизнес-логику и работает с базой данных.
Точка API	Конкретный HTTP-маршрут, через который клиентская часть обращается к серверному приложению.
Исходные замеры	Контрольная точка до изменений: результаты тестов, время ответа, ошибки и нагрузочные показатели.
Очередь задач	Механизм, при котором приложение передает работу в очередь, а отдельный рабочий процесс выполняет ее фоном.
Рабочий процесс	Отдельный процесс, который забирает фоновые задачи из очереди и выполняет тяжелые операции вне основного HTTP-запроса.
Развертывание	Описание запуска и поддержания нужного числа экземпляров приложения в Kubernetes.
Служба	Стабильная точка доступа к группе pod в Kubernetes.
Карта настроек	Объект Kubernetes для хранения неконфиденциальных параметров конфигурации.
Горизонтальное ав- томасштабирование	Автоматическое изменение числа экземпляров приложения в зависимости от нагрузки.
Метрика	Измеряемый показатель работы системы: количество запросов, время ответа, число ошибок или глубина очереди.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Официальная документация Python. Электронный ресурс. URL: <https://docs.python.org/3/>.
2. Официальная документация FastAPI. Электронный ресурс. URL: <https://fastapi.tiangolo.com/>.
3. Официальная документация SQLAlchemy. Электронный ресурс. URL: <https://docs.sqlalchemy.org/>.
4. Официальная документация PostgreSQL. Электронный ресурс. URL: <https://www.postgresql.org/docs/>.
5. Официальная документация Redis. Электронный ресурс. URL: <https://redis.io/docs/>.
6. Официальная документация Celery. Электронный ресурс. URL: <https://docs.celeryq.dev/>.
7. Официальная документация Docker. Электронный ресурс. URL: <https://docs.docker.com/>.
8. Официальная документация Kubernetes. Электронный ресурс. URL: <https://kubernetes.io/docs/>.
9. Официальная документация Prometheus. Электронный ресурс. URL: <https://prometheus.io/docs/>.
10. Официальная документация Nginx. Электронный ресурс. URL: <https://nginx.org/en/docs/>.

ПРИЛОЖЕНИЕ

6 Рабочий план-график прохождения практики

№ п/ п	Сроки проведе- ния	Планируемые работы	Отметка
1	01.07.2026	Инструктаж по ознакомлению с требованиями охраны труда, техники безопасности, пожарной безопасности, а также правилами внутреннего трудового распорядка.	Выполне- но
2	02.07.2026 – 03.07.2026	Изучение материалов по Kubernetes, Celery и Prometheus. Анализ текущей серверной архитектуры и фиксация исходных замеров.	Выполне- но
3	04.07.2026 – 06.07.2026	Подготовка серверной инфраструктуры к запуску в Kubernetes: Deployment, Service, ConfigMap, Secret, проверки состояния, задача миграции.	Выполне- но
4	07.07.2026 – 09.07.2026	Внедрение Celery для фоновых задач: настройка посредника сообщений, рабочего процесса, политики повторного запуска, очередей и дополнительных фоновых операций.	Выполне- но
5	10.07.2026 – 12.07.2026	Добавление Prometheus-метрик для API, очередей и фоновых задач. Подготовка базовых команд проверки метрик.	Выполне- но
6	13.07.2026 – 14.07.2026	Проведение нагрузочного тестирования, проверка НРА, отказоустойчивости pod и подготовка итоговой документации.	Выполне- но